Meeting ?? SP23

Out of Order Execution

@crawfish

Outline

Background

Out of Order Execution

Section 1

Background

```
LOAD R0, [LABEL1]
ADD R1, R2, R3
AND R4, R5, R6
XOR R7, R7, R7
NOP ; t = ???
```

Pipelining

- Designed to improve throughput of CPU

Pipelining

```
LOAD R0, [LABEL1]
ADD R1, R2, R3
AND R4, R5, R6
XOR R7, R7, R7
NOP ; t = ???
```

Parallel Execution

- Also increases instruction throughput
- Different kinds of parallelism
  - Thread level (GPU)
  - Instruction level

Instruction Level Parallelism

```
LOAD R0, [LABEL1]
ADD R1, R2, R3
AND R4, R5, R6
XOR R7, R7, R7
NOP ; t = ???
```

Section 2

Out of Order Execution

Out of Order Execution

- Technique to exploit instruction level parallelism at the CPU level
- Re-order instructions on CPU during code execution based on dependencies
  - Why don't we make the compiler do this?
- 'Stylized pipelining'
  - Execution of a single instruction is still broken into parts
- Introduces a *lot* of complexity

Scoreboarding

- Simple form of OOO execution
- First used in the CDC 6600 (Sept. 1964)
- Based on 'scoreboard' that manages dependencies
  - Lists dependencies of currently executing instructions

Scoreboarding

1. Fetch / Decode
2. Issue
3. Read Operands
4. Execution
5. Write-Back

Scoreboarding

1. Fetch / Decode
2. Issue
   - Issued directly to a computation unit
3. Read Operands
4. Execution
5. Write-Back

Scoreboarding

1. Fetch / Decode
2. Issue
3. Read Operands
   - Computation unit waits until all operands are available
   - Reads operands (inputs)
4. Execution
5. Write-Back

Scoreboarding

1. Fetch / Decode
2. Issue
3. Read Operands
4. Execution
   - Occurs once all operands are fetched
   - Notifies 'scoreboard' of result
5. Write-Back

Scoreboarding

1. Fetch / Decode
2. Issue
3. Read Operands
4. Execution
5. Write-Back
   - Completed execution results are taken and written to destination registers (if available)

Data Hazards

- Data hazards are instruction dependencies with the potential to result in incorrect execution
- Pipelining introduced RAW hazards
- OOO execution provides opportunity for other types of data hazards

Data Hazards

- Data hazards are instruction dependencies with the potential to result in incorrect execution
- Pipelining introduced RAW hazards
- OOO execution provides opportunity for other types of data hazards
  - WAR, WAW hazards
  - Why weren't these a problem before? / Why are they a problem now?

WAR

```
LOAD R2, [LABEL]
ADD R0, R1, R2
ADD R1, R7, R7
NOP
NOP
NOP ; R0 = ???
```

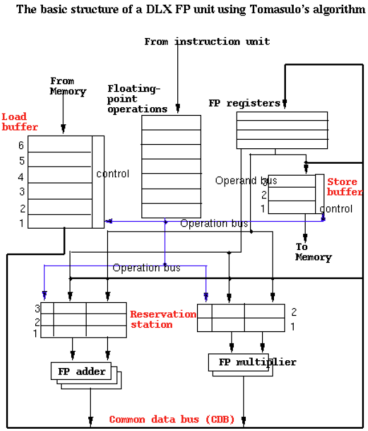
WAW

```
LOAD R0, [LABEL]
ADD R0, R1, R2
NOP
NOP
NOP ; R0 = ???
```

Tomasulo's Algorithm

- Hardware algorithm for scheduling of instructions
- Developed at IBM in 1967
- Key contributions
  1. Register renaming
  2. Speculative execution
  3. ROB (Re-Order Buffer)
- Issue stage targets reservation stations instead of execution units

IBM FP Unit


The basic structure of a DLX FP unit using Tomasulo's algorithm
From instruction unit
From Memory
Floating-point operations
FP registers
Load buffer
6
5
4
3
2
1
control
Operand bus
Store buffer
2
1
control
Operation bus
To Memory
Operation bus
3
2
1
Reservation station
2
1
FP adder
FP multiplier
Common data bus (CDB)

Register Renaming

```
XOR R4, R4, R4
ADD R4, R4, R5
ADD R4, R4, R6
LOAD R4, [LABEL] ; Should this execute in order?
```

Register Renaming

- WAR and WAW are both *artificial* dependencies

Register Renaming

- WAR and WAW are both *artificial* dependencies
  - Using a different register as the destination could solve our problems

Register Renaming

- WAR and WAW are both *artificial* dependencies
  - Using a different register as the destination could solve our problems
- When a register is designated as a destination, it does not matter which register we pick
  - Contents is about to be overwritten
- Abstract logical register names from hardware registers, allow destination register to be assigned based on what is currently 'free'

Speculative Execution

```
LOAD R2, [LABEL]
BNE R2, R0, PLACE
ADD R1, R1, R2
PLACE:
ADD R1, R1, R3
```

Speculative Execution

- We've seen this before in pipelining
  - How do we avoid stalling when we have conditional instructions?
- Scoreboarding method stalls on any branches, refuses to act speculatively
- Tomasulo algorithm will 'guess' using a branch predictor
  - Incorrectly predicted branches will have instructions flushed from pipeline before commit

Re-Order Buffer

- Since instructions are being executed out of order, results may not be assembled correctly
- Some workloads demand support for precise interrupts
  - How do we ensure that the register file reflects the correct state of the processor at every timestep
- Re-order instructions at commit

The end.